Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información

Programación 2 · Sección A

Proyecto CarritoGT

Migración de un carrito de compras en PHP a una API REST con Spring Boot

Documento técnico

Ingeniero: Moises Eduardo Velasquez Oliva

Estudiante: Jesus Hernandez Alonzo

Carné: 3490-25-3298

Repositorio: github.com/jherandeza-code/proyecto-carritogt

Guatemala, 17 de octubre de 2026

Índice

1. Portada 3
2. Evidencia de virtualización 3
3. Justificación de recursos 4
4. Evidencia del servidor 4
5. Configuración del Virtual Host 6
6. Diagrama de casos de uso 7
7. Diagramas de arquitectura 8
8. Diccionario de datos 12
9. Contrato de la API 13
10. Evidencia de pruebas 13
11. Manual de usuario 16
12. Bitácora de desarrollo 17

1. Portada

La portada está en la primera página de este documento.

2. Evidencia de virtualización

Todo el proyecto lo trabajé dentro de una máquina virtual en VirtualBox, con Lubuntu 22.04 como sistema invitado. Mi laptop tiene Windows 11 y la máquina virtual corre al mismo tiempo en una ventana aparte.

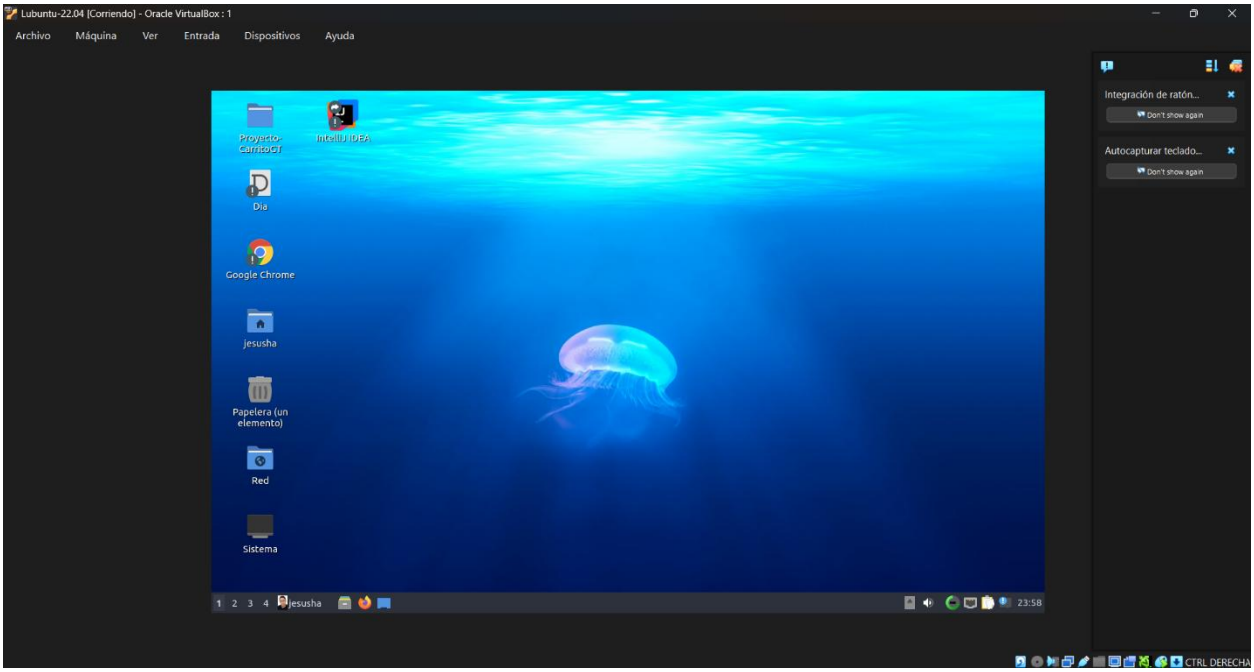


Figura 1. Máquina virtual Lubuntu-22.04 corriendo en Oracle VirtualBox sobre Windows.

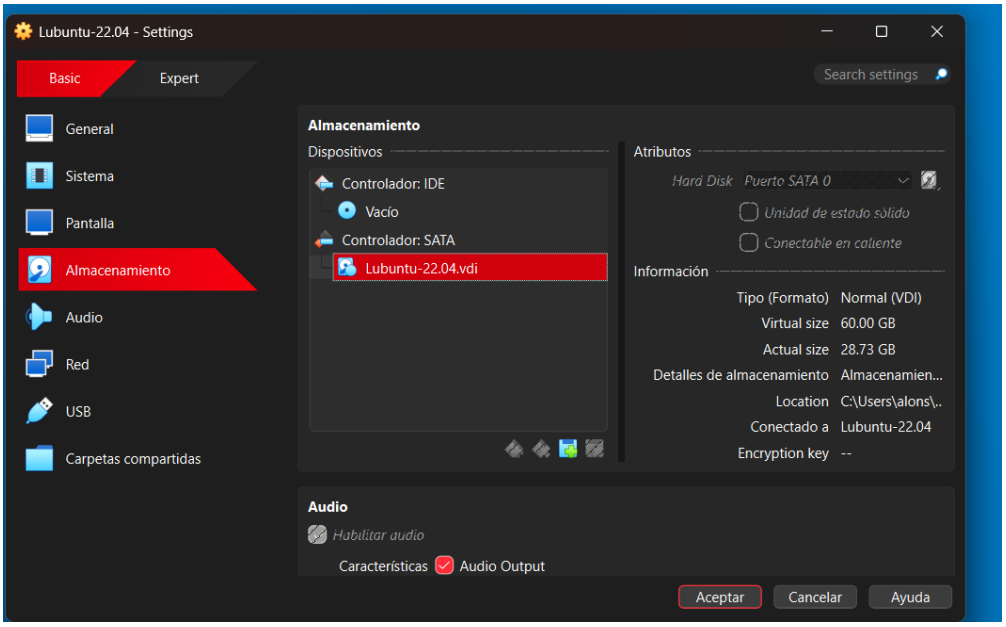


Figura 2. Configuración de almacenamiento de la máquina virtual (disco virtual de 60 GB).

3. Justificación de recursos

Características de mi equipo (anfitrión):



Figura 3. Características de la laptop: Intel Core i5-1235U, 16 GB de RAM y 477 GB de almacenamiento.

Recursos que le asigné a la máquina virtual y por qué:

Recurso	Asignado	Por qué
Sistema operativo	Lubuntu 22.04 (64 bits)	Es una versión ligera de Ubuntu; deja más memoria libre para IntelliJ, MySQL y Spring Boot.
Memoria RAM	8 GB de 16 GB	La mitad de la RAM: alcanza para IntelliJ, la API y MySQL al mismo tiempo, y Windows se queda con 8 GB.
Procesadores	4 de 12 hilos	Al principio tenía 2 y compilar con Maven era lento; con 4 la VM responde mejor sin dejar sin recursos a Windows.
Disco	60 GB (dinámico)	Empecé con 30 GB y llegó al 87 % por IntelliJ y las librerías de Maven; lo amplié a 60 GB. Al ser dinámico solo ocupa lo que se usa.
Memoria de video	128 MB	Para que el escritorio y el navegador se vean fluidos.
Red	NAT	La VM sale a internet (apt, Maven, GitHub) y todos los servicios se usan dentro de la misma máquina con localhost.

4. Evidencia del servidor

En la máquina virtual instalé y configuré el servidor con Apache, PHP y MySQL, además de Java 17 para Spring Boot. En la captura se ven las versiones y que Apache y MySQL están activos.

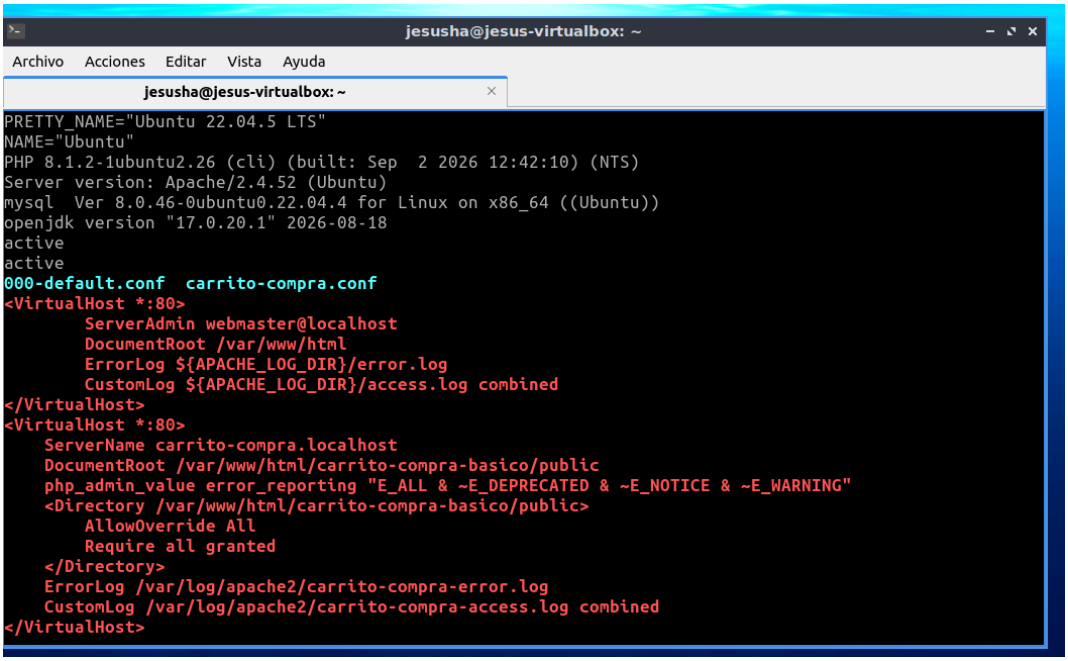


Figura 4. Versiones: Ubuntu 22.04.5, PHP 8.1.2, Apache 2.4.52, MySQL 8.0.46, OpenJDK 17.0.20; apache2 y mysql activos.

Componente	Versión	Uso en el proyecto
Apache	2.4.52	Servidor web de la versión PHP
PHP	8.1.2	Ejecuta la versión original del carrito
MySQL	8.0.46	Base de datos cc (usada por las dos versiones)
OpenJDK	17.0.20	Ejecuta la API en Spring Boot
Git	—	Control de versiones y subida a GitHub

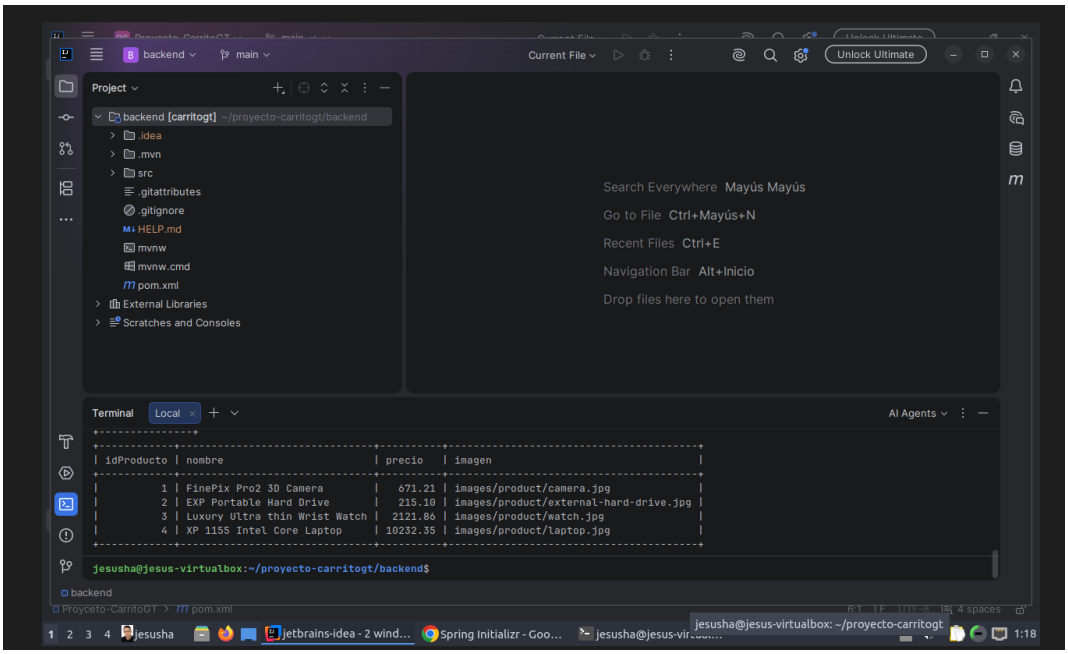


Figura 5. Base de datos cc en MySQL con los 4 productos de prueba.

5. Configuración del Virtual Host

Para la versión PHP creé el archivo /etc/apache2/sites-available/carrito-compra.conf y lo habilité. El DocumentRoot apunta a la carpeta public del proyecto, donde está el index.php que recibe todas las peticiones:

```
<VirtualHost *:80>
    ServerName carrito-compra.localhost
    DocumentRoot /var/www/html/carrito-compra-basico/public
    <Directory /var/www/html/carrito-compra-basico/public>
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog /var/log/apache2/carrito-compra-error.log
    CustomLog /var/log/apache2/carrito-compra-access.log combined
</VirtualHost>
```

AllowOverride All es necesario para que funcione el archivo .htaccess, que manda todas las rutas a public/index.php. En /etc/hosts agregué la línea 127.0.0.1 carrito-compra.localhost para que el nombre apunte a la misma máquina.

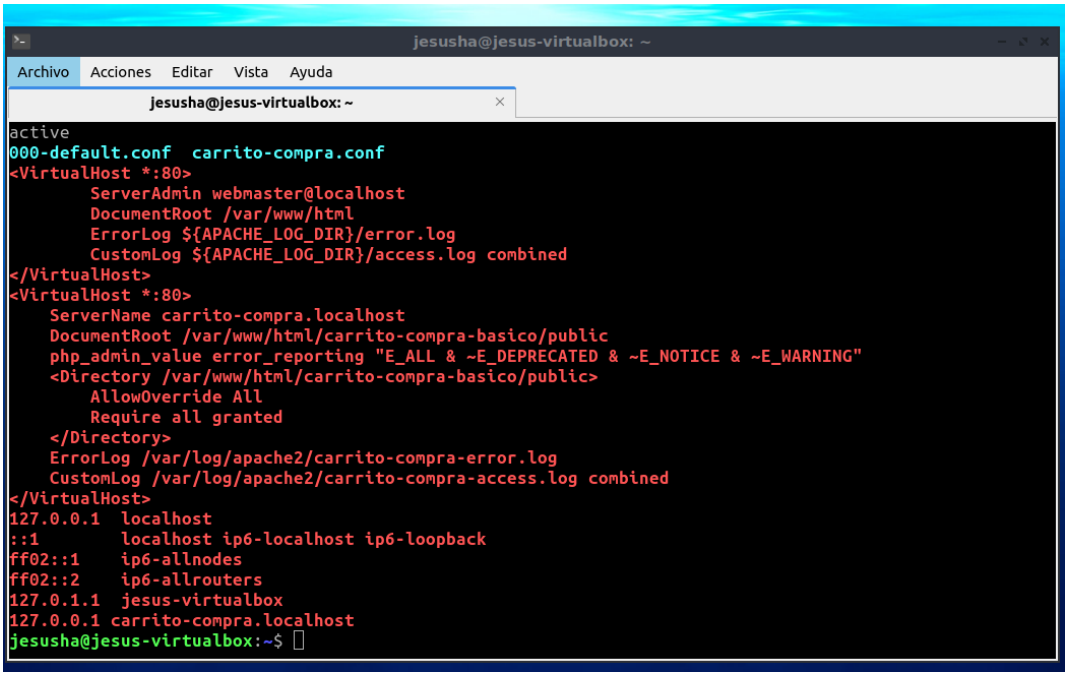


Figura 6. Archivo del Virtual Host habilitado y /etc/hosts con carrito-compra.localhost.

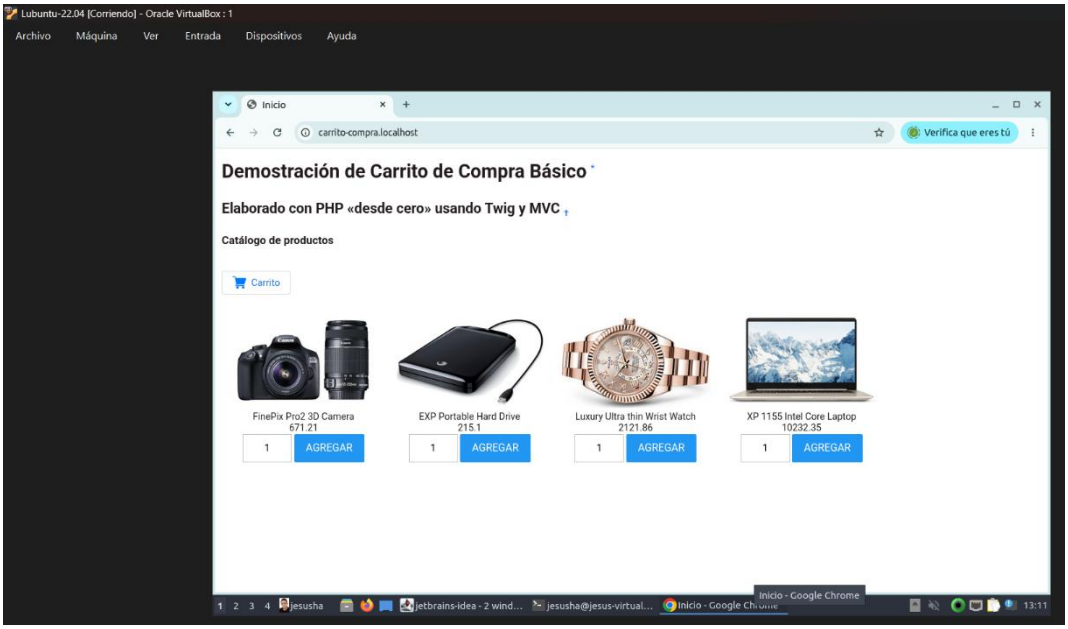


Figura 7. Versión PHP funcionando en http://carrito-compra.localhost (catálogo).

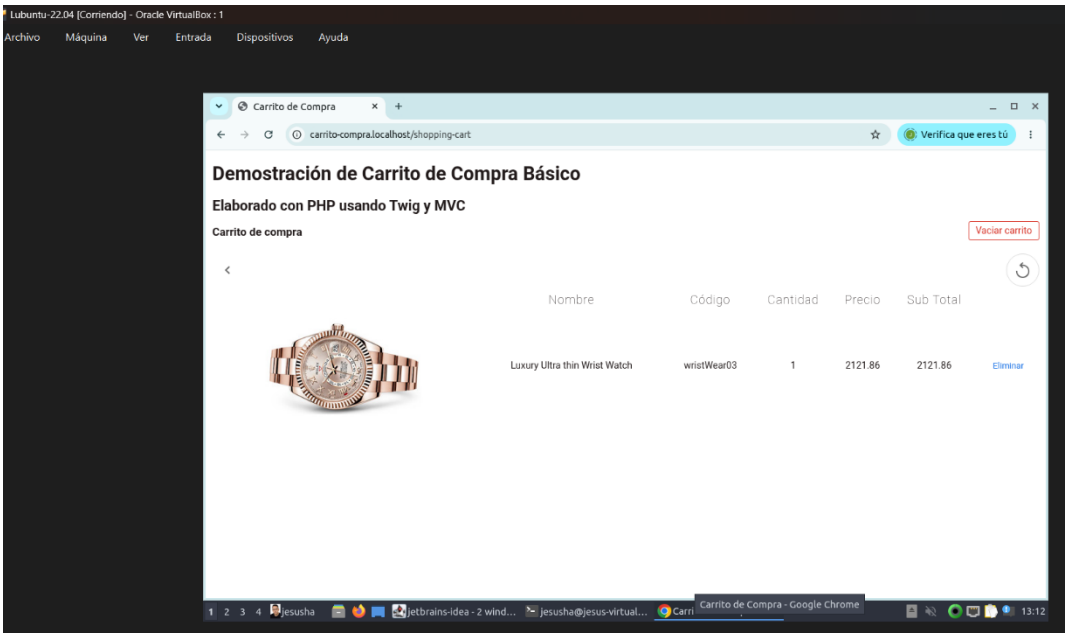


Figura 8. Versión PHP: carrito de compra.

6. Diagrama de casos de uso

El actor principal es el Cliente, que visita la tienda. Dentro del límite del sistema CarritoGT están las cuatro funciones que puede usar. Cada caso de uso corresponde a un endpoint de la API.

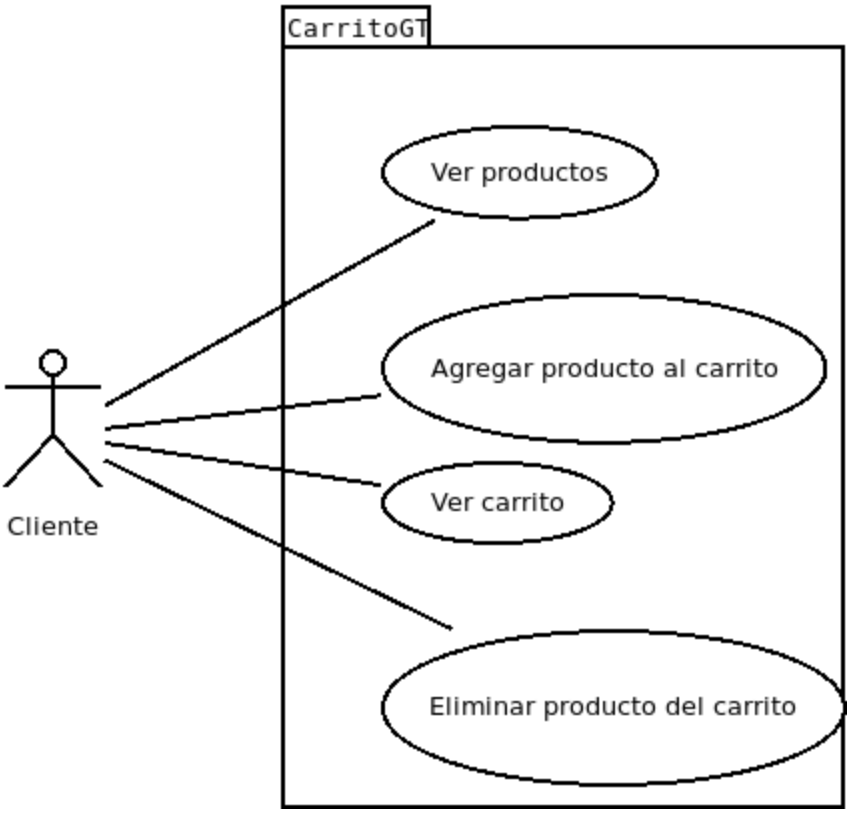


Figura 9. Diagrama de casos de uso de CarritoGT.

Caso de uso	Requerimiento	Endpoint
Ver productos	RF-01, RF-02	GET /api/productos
Agregar producto al carrito	RF-03	POST /api/carrito
Ver carrito	RF-04	GET /api/carrito
Eliminar producto del carrito	RF-05	DELETE /api/carrito/{id}

7. Diagramas de arquitectura

7.1 Versión PHP

La versión PHP es una aplicación MVC monolítica: Apache recibe la petición, el Router elige el controlador, el modelo consulta MySQL con PDO y Twig arma la página HTML completa en el servidor.

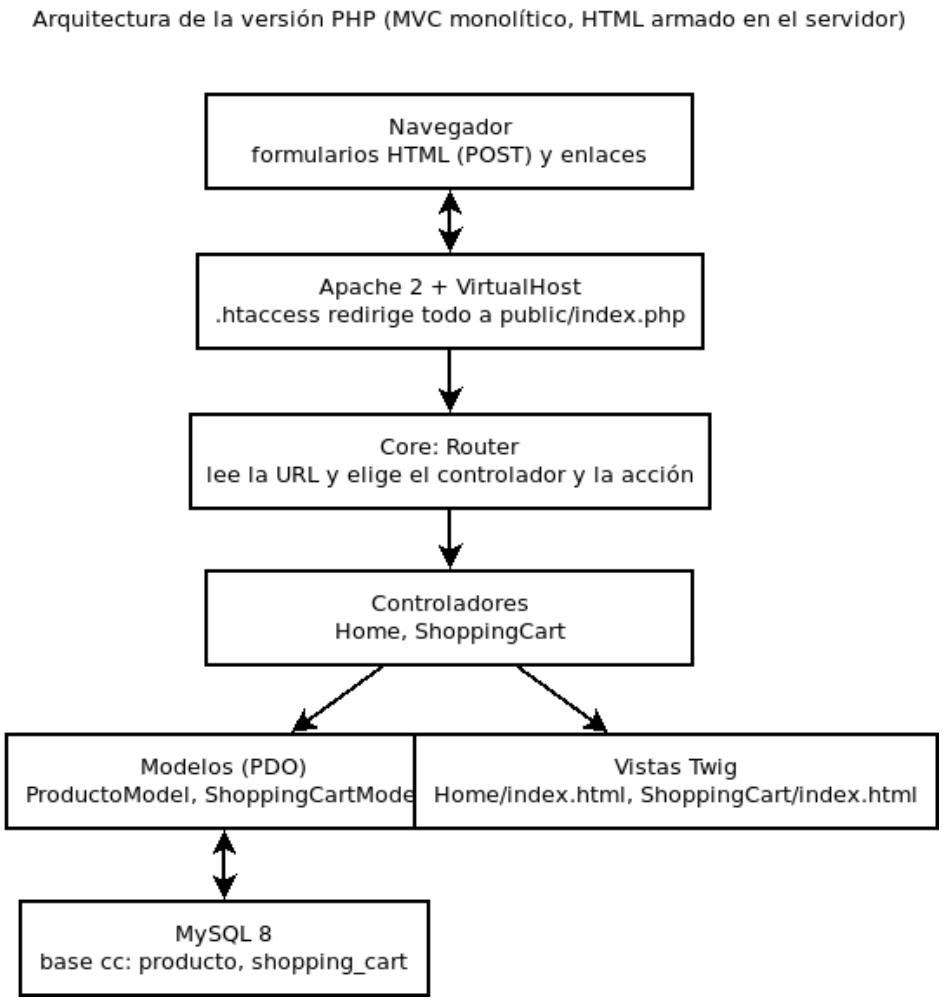


Figura 10. Arquitectura de la versión PHP.

Diagrama de clases de la versión PHP, hecho a partir del análisis del código (capas core, controllers, models y app):

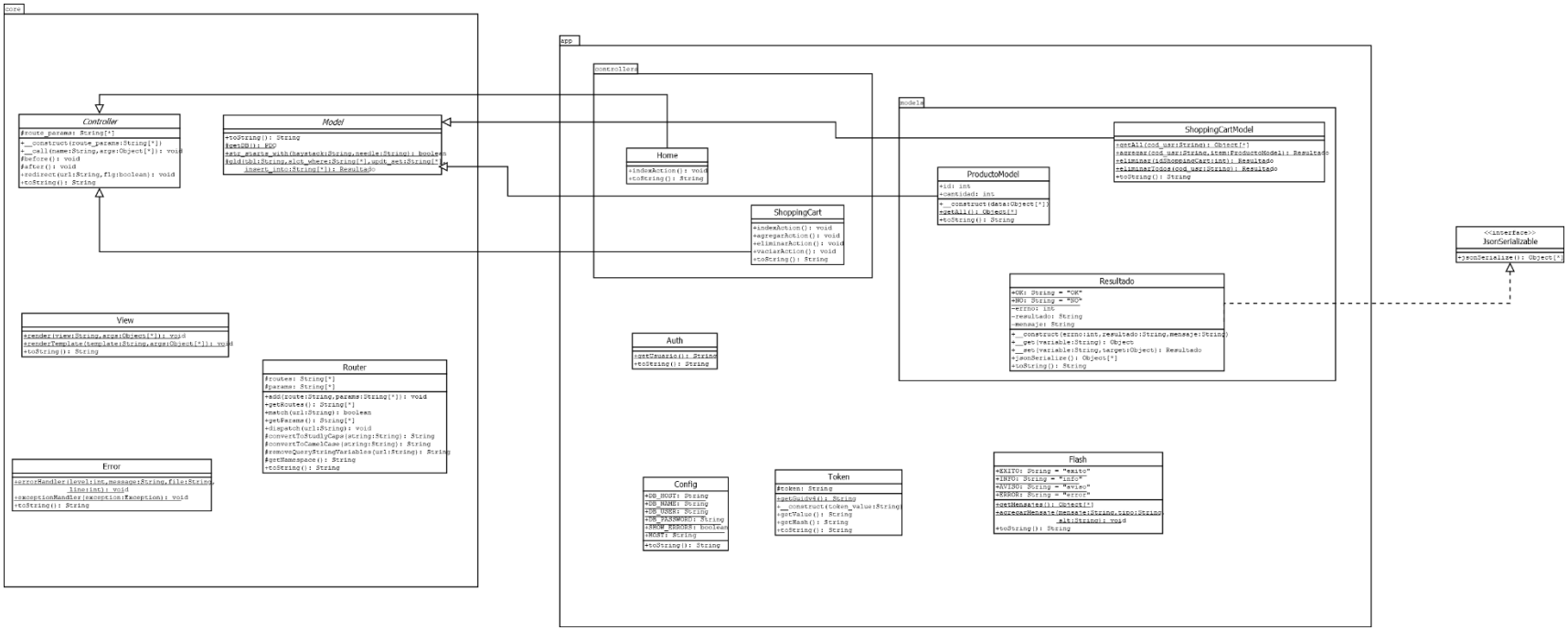

core
Controller
#route_params: String[*]
+__construct(route_params:String[*])
+__call(name:String,args:Object[*]): void
#before(): void
#after(): void
+redirect(url:String,flg:boolean): void
+toString(): String
Model
+toString(): String
#getDB(): PDO
#str_starts_with(haystack:String,needle:String): boolean
#guildbl:String,sql_where:String[*],sqot_set:String[*],
insert_into:String[*]): ResultaDo
View
+render(view:String,args:Object[*]): void
+renderTemplate(template:String,args:Object[*]): void
+toString(): String
Router
#routes: String[*]
#params: String[*]
+add(route:String,params:String[*]): void
+getRoutes(): String[*]
+match(url:String): boolean
+getParams(): String[*]
+dispatch(url:String): void
#convertToStudlyCaps(string:String): String
#convertToCamelCase(string:String): String
#removeQueryStringVariables(url:String): String
#getNamespace(): String
+toString(): String
Error
+errorHandler(level:int,message:String,file:String,
line:int): void
+exceptionHandler(exception:Exception): void
+toString(): String
app
controllers
Home
+indexAction(): void
+toString(): String
ShoppingCart
+indexAction(): void
+agregarAction(): void
+eliminarAction(): void
+vaciarAction(): void
+toString(): String
Auth
+getUsuario(): String
+toString(): String
models
ShoppingCartModel
+getAll(cod_usr:String): Object[*]
+agregar(cod_usr:String,item:ProductoModel): Resultado
+eliminar(idShoppingCart:int): Resultado
+eliminarTodo(cod_usr:String): Resultado
+toString(): String
ProductoModel
+id: int
+cantidad: int
+__construct(data:Object[*])
+getAll(): Object[*]
+toString(): String
Resultado
+OK: String = "OK"
+NO: String = "NO"
-error: int
-resultado: String
-mensaje: String
+__construct(error:int,resultado:String,mensaje:String)
+__get(variable:String): Object
+__set(variable:String,target:Object): Resultado
+jsonSerialize(): Object[*]
+toString(): String
<<interface>>
JsonSerializable
+jsonSerialize(): Object[*]
Config
+DB_HOST: String
+DB_NAME: String
+DB_USER: String
+DB_PASSWORD: String
+SHOW_ERRORS: boolean
-HOST: String
+toString(): String
Token
#token: String
+getGuidv4(): String
+__construct(token_value:String)
+getValue(): String
+getHash(): String
+toString(): String
Flash
+EXITO: String = "exito"
+INFO: String = "info"
+AVISO: String = "aviso"
+ERROR: String = "error"
+getMensaje(): Object[]
+agregarMensaje(mensaje:String,tipo:String)
alt:String): void
+toString(): String

Figura 11. Diagrama de clases de la versión PHP (girado para que se lea mejor).

7.2 Versión Spring Boot por capas

La versión nueva sigue siendo un monolito, pero organizado en capas. El frontend (HTML y JavaScript) ya no recibe páginas armadas: pide datos en JSON a la API con fetch.

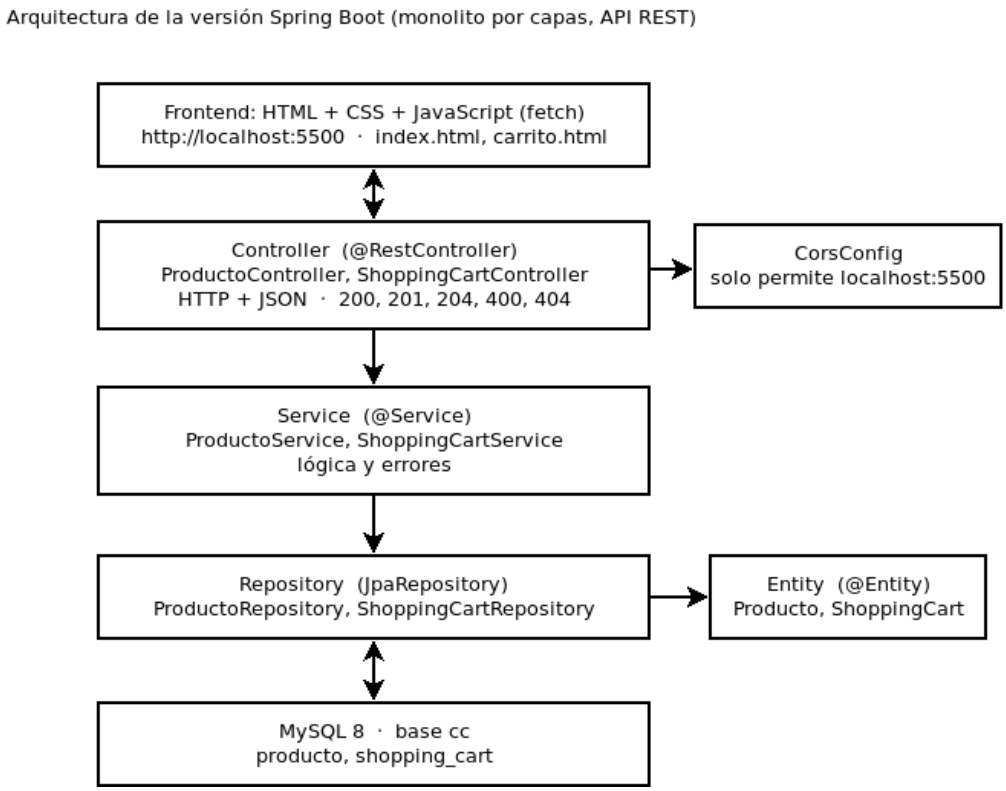


Figura 12. Arquitectura de la versión Spring Boot por capas.

Capa	Clases	Responsabilidad
Controller	ProductoController, ShoppingCartController	Recibe las peticiones HTTP y responde JSON con el código HTTP correcto.
Service	ProductoService, ShoppingCartService	Lógica del negocio y validaciones (por ejemplo, si el producto existe).
Repository	ProductoRepository, ShoppingCartRepository	Acceso a la base de datos con Spring Data JPA, sin escribir SQL a mano.
Entity	Producto, ShoppingCart	Representan las tablas producto y shopping_cart.
Config	CorsConfig	Permite que el frontend (localhost:5500) consuma la API (localhost:8080).

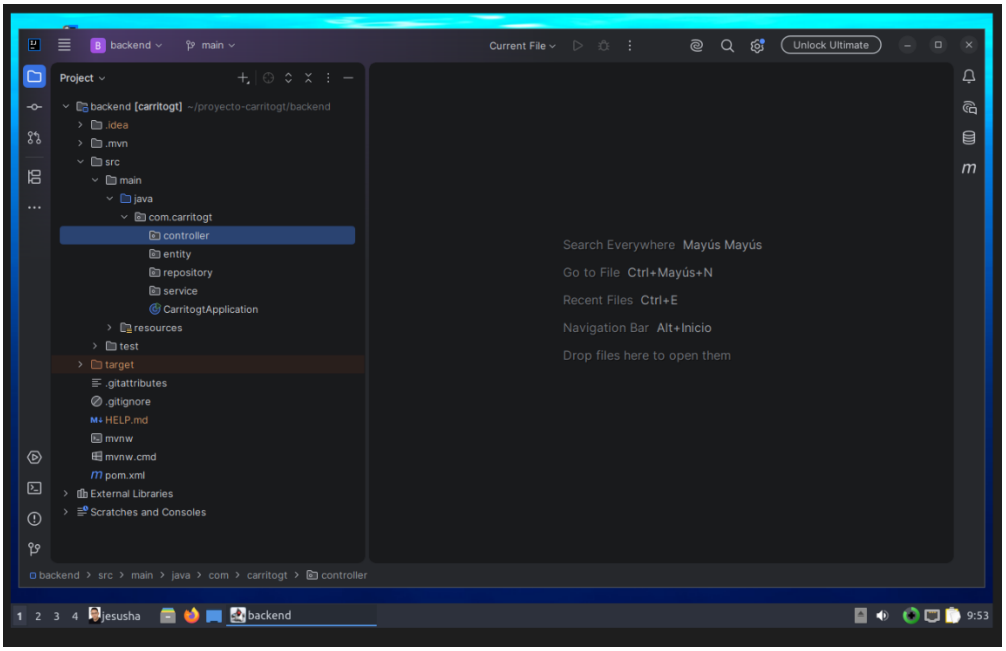


Figura 13. Paquetes del proyecto Spring Boot en IntelliJ, uno por capa.

8. Diccionario de datos

Base de datos: cc (motor InnoDB, juego de caracteres utf8, collation utf8_spanish_ci). Se usa el esquema de dos tablas del script del ingeniero, sin cambios (database/cc.sql).

Tabla producto

Campo	Tipo	Nulo	Llave	Descripción
idProducto	INT(11)	No	PK, AUTO_INCREMENT	Identificador del producto
nombre	VARCHAR(255)	Sí		Nombre que se muestra en el catálogo
codigo	VARCHAR(255)	No		Código interno del producto
imagen	TEXT	Sí		Ruta o URL de la imagen del producto
precio	DOUBLE(10,2)	Sí		Precio unitario

Tabla shopping_cart

Campo	Tipo	Nulo	Llave	Descripción
idShoppingCart	INT(11)	No	PK, AUTO_INCREMENT	Identificador del ítem del carrito
idProducto	INT(11)	Sí	FK → producto.idProducto	Producto agregado al carrito
codUsuario	VARCHAR(128)	Sí		Código del visitante dueño del carrito
cantidad	INT(11)	Sí		Cantidad del producto

Relación: un producto puede estar en muchos ítems de carrito y cada ítem apunta a un solo producto (1 a muchos), con la llave foránea cc_profk_1. Por esa llave, MySQL no deja borrar un producto que esté en algún carrito.

9. Contrato de la API

Dirección base: http://localhost:8080/api. Todas las respuestas son JSON.

Endpoint	Método	Propósito	Parámetros	Respuestas
/api/productos	GET	Listar los productos	Ninguno	200: lista de productos
/api/carrito	GET	Ver el carrito de un visitante	usuario (query, obligatorio)	200: lista de ítems; 400 si falta usuario
/api/carrito	POST	Agregar un producto al carrito	usuario e idProducto (query, obligatorios)	201: ítem creado; 404 si el producto no existe; 400 si falta un parámetro
/api/carrito/{id}	DELETE	Eliminar un ítem del carrito	id del ítem (ruta)	204: eliminado; 404 si el ítem no existe

Ejemplo de respuesta de GET /api/productos (200):

```
[{"idProducto":1,"nombre":"FinePix Pro2 3D Camera","codigo":"3DcAM01",
  "imagen":"images/product/camera.jpg","precio":671.21}, ...]
```

Ejemplo de respuesta de POST /api/carrito?usuario=prueba&idProducto=1 (201):

```
{"idShoppingCart":4,"codUsuario":"prueba","cantidad":1,
 "producto":{"idProducto":1,"nombre":"FinePix Pro2 3D Camera","codigo":"3DcAM01",
             "imagen":"images/product/camera.jpg","precio":671.21}}
```

Ejemplo de error, POST con idProducto=99 (404):

```
{"timestamp":"2026-10-17T16:33:15.432Z","status":404,"error":"Not
Found","path":"/api/carrito"}
```

Con la propiedad server.error.include-message=always en application.properties, la respuesta también trae el campo "message" con el texto del error (por ejemplo, "El producto 99 no existe").

10. Evidencia de pruebas

Probé cada endpoint con cURL desde la terminal antes de hacer el frontend, y después probé la aplicación completa en el navegador.

Prueba	Esperado	Resultado
Arrancar la API	Started CarritogtApplication	Correcto (figura 14)
GET /api/productos	200 y 4 productos	Correcto (figura 15)
POST /api/carrito con idProducto=1	201 y el ítem creado	Correcto (figura 16)

Prueba	Esperado	Resultado
GET /api/carrito?usuario=prueba	200 y la cámara en el carrito	Correcto (figura 17)
POST con idProducto=99	404 sin que la API se caiga	Correcto (figura 18)
DELETE /api/carrito/4	204	Correcto (figura 19)
Frontend: catálogo y agregar	Productos con imagen y aviso de agregado	Correcto (figura 20)
Frontend: ver carrito	Tabla con los productos agregados	Correcto (figura 21)

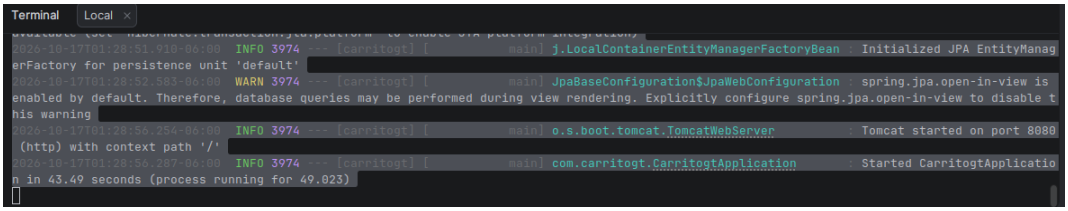


Figura 14. La API arranca en el puerto 8080 conectada a MySQL.

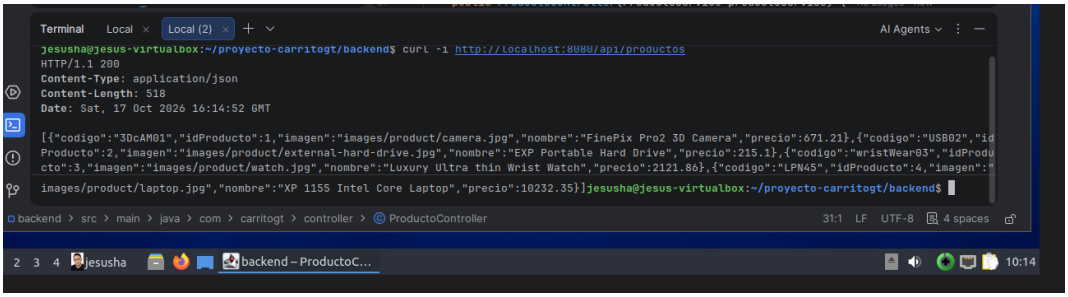


Figura 15. GET /api/productos responde 200 con JSON.

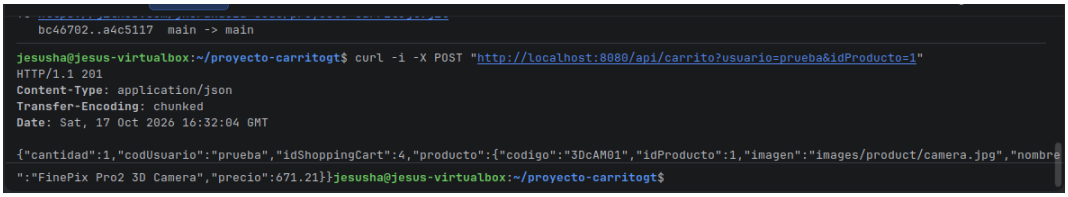


Figura 16. POST /api/carrito responde 201 Created.

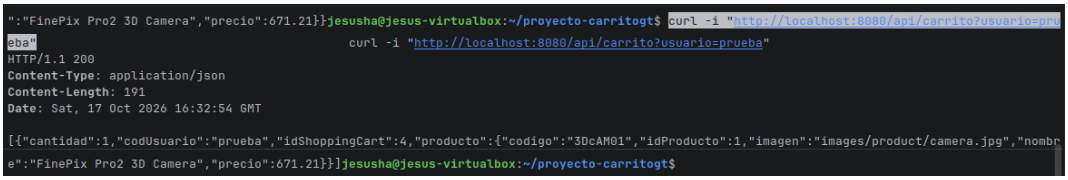


Figura 17. GET /api/carrito responde 200 con el carrito del usuario.



Figura 18. Un producto que no existe responde 404 (manejo de errores, RF-08).

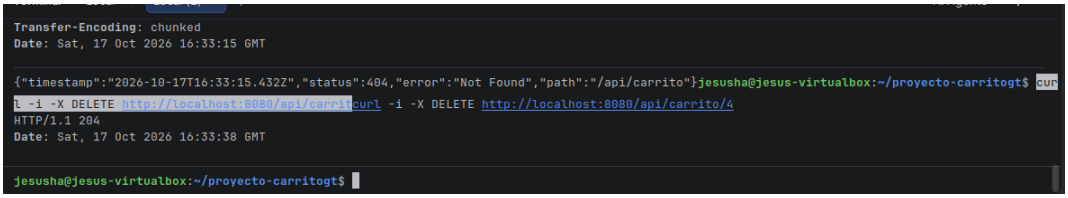


Figura 19. DELETE /api/carrito/4 responde 204 No Content.

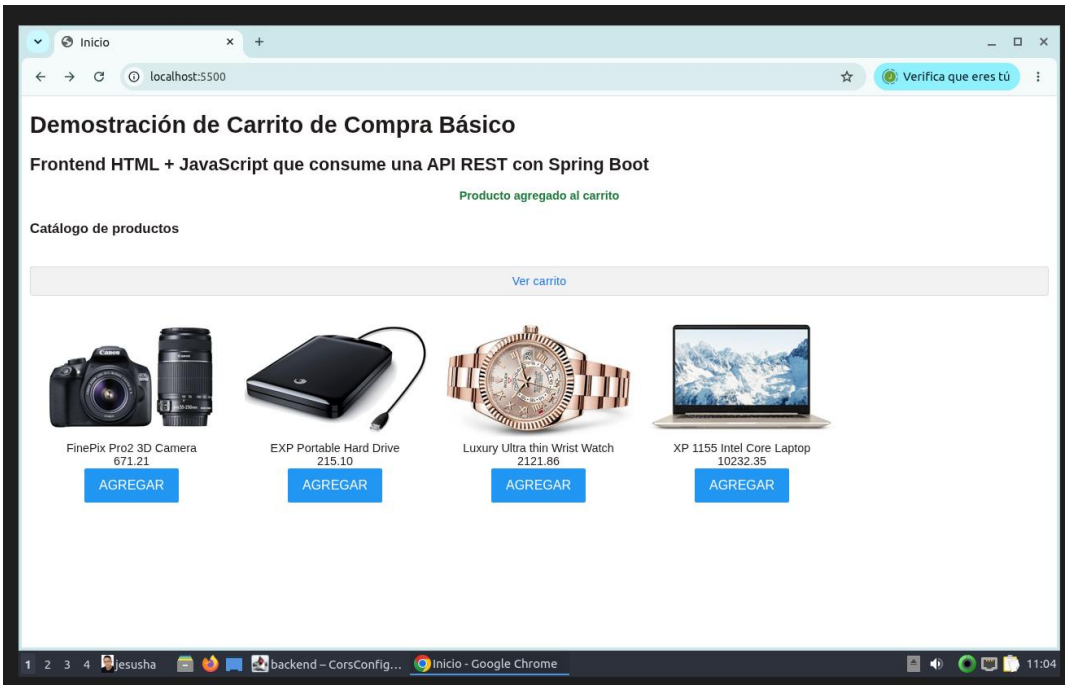


Figura 20. Frontend consumiendo la API: catálogo y mensaje de producto agregado.

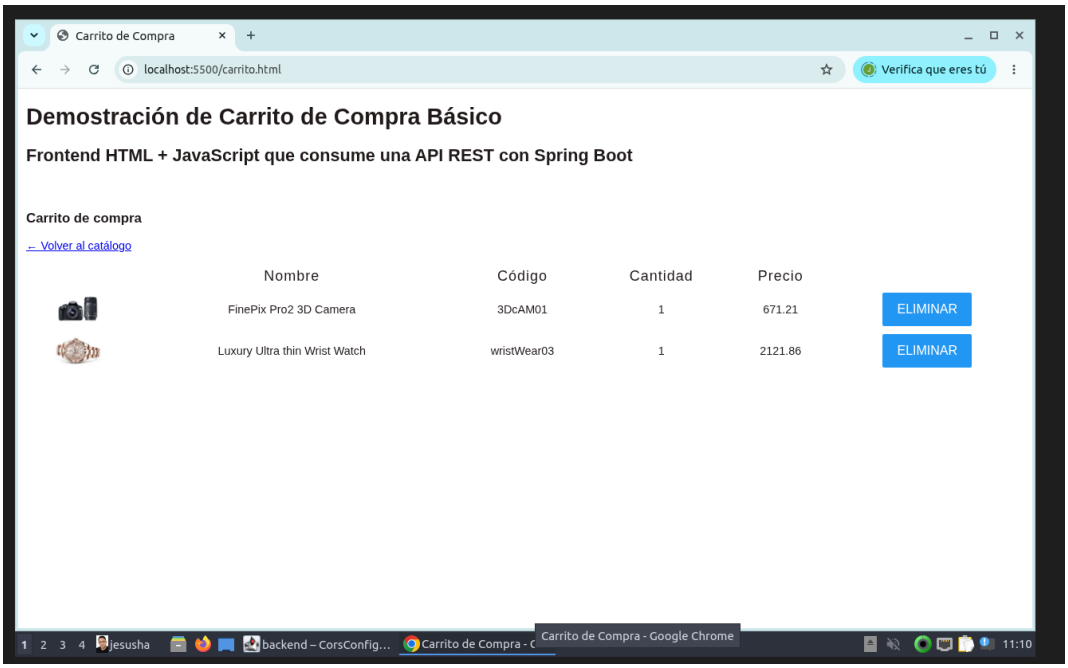


Figura 21. Frontend: carrito con los productos agregados.

11. Manual de usuario

Antes de empezar

Se necesitan dos terminales abiertas en la máquina virtual:

```
# Terminal 1: la API
cd ~/proyecto-carritogt/backend
./mvnw spring-boot:run

# Terminal 2: el frontend
cd ~/proyecto-carritogt/frontend
python3 -m http.server 5500
```

Uso de la tienda

- Abrir el navegador en http://localhost:5500. Aparece el catálogo con los productos, su imagen y su precio.
- Presionar AGREGAR en un producto. Arriba aparece el mensaje en verde «Producto agregado al carrito».
- Presionar «Ver carrito» para ver la tabla con los productos agregados (imagen, nombre, código, cantidad y precio).
- Presionar ELIMINAR en una fila para quitar ese producto. La tabla se actualiza sola.
- Si el carrito no tiene productos aparece «Tu carrito de compra está vacío».
- Con «Volver al catálogo» se regresa a la página de productos.
- Si la API no está encendida o hay un error, la página muestra un mensaje en rojo y no se traba.

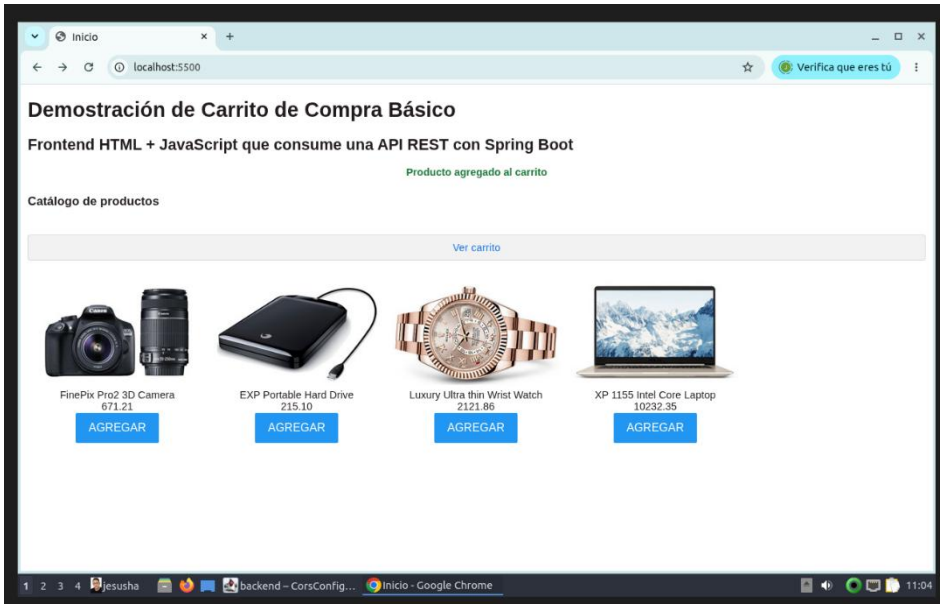


Figura 22. Catálogo de productos.

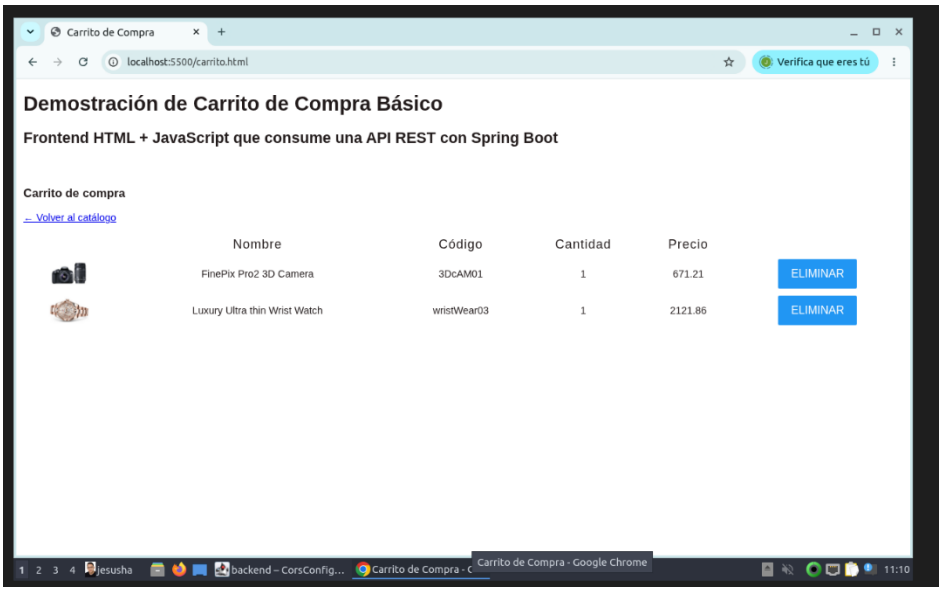


Figura 23. Carrito de compra.

12. Bitácora de desarrollo

Fecha	Avance realizado	Problema o decisión	Solución / aprendizaje	Commit
12/10/2026	Creé el repositorio en GitHub y lo conecté con IntelliJ	Al principio hice un proyecto Maven normal, pero no tenía Spring Boot	Lo dejé en la raíz como mi primer intento; el proyecto que sí funciona está en la carpeta backend/	d837726 Primer commit, conexión a github
15/10/2026	Revisé el código PHP del ingeniero y empecé el diagrama de clases en Dia con la capa core	No sabía quitar la negrita de los nombres ni cómo marcar los métodos estáticos	En la pestaña Estilo cambié a Normal, y con "Vista de clase" el método sale subrayado como estático	9d549fb docs: agrega capa core al diagrama de clases de la version PHP
15/10/2026	Agregué las capas controllers y models al diagrama	ProductoModel no tiene atributos escritos, los crea en el constructor con un foreach	Puse los atributos que de verdad le llegan del formulario: id y cantidad	5da3a82 docs: agrega capas controllers y models al diagrama de clases PHP
15/10/2026	Terminé las clases de la capa app (Auth, Config, Flash, Token)	No sabía cómo acomodar las capas	Las ordené según el namespace de cada archivo; controllers y models quedaron dentro de app	dadc464 docs: agrega clases de la capa app al diagrama de clases PHP
16/10/2026	Le di más recursos a la máquina virtual	La VM iba lenta y se trabó al sacar una instantánea	La apagué y le subí a 4 procesadores y 60 GB de disco	—
17/10/2026	Creé el proyecto Spring Boot con Spring Initializr (Java 17, Maven, Spring Web, JPA y MySQL)	El disco de la VM ya estaba casi lleno (87 %)	Agrandé la partición para usar los 60 GB y generé el proyecto dentro de backend/	bc46702 chore: genera proyecto Spring Boot con Spring Initializr
17/10/2026	Conecté la API a MySQL e hice el endpoint GET /api/productos con sus 4 capas; también terminé las flechas del diagrama	No podía entrar con el usuario root de MySQL, y JPA buscaba la columna id_producto en lugar de idProducto	Creé un usuario carrito solo con los permisos necesarios, y configuré JPA para que no cambie las tablas ni los nombres	a4c5117 feat: agrega endpoint GET /api/productos con capas Entity, Repository, Service y Controller
17/10/2026	Hice los endpoints del carrito (ver, agregar y eliminar) y los probé con cURL	En una API no hay sesión como en PHP, entonces ¿cómo sé de quién es el carrito?	Mando el código del usuario en cada petición; si el producto no existe responde 404	ace615e feat: agrega endpoints del carrito GET, POST y DELETE

Fecha	Avance realizado	Problema o decisión	Solución / aprendizaje	Commit
				/api/carrito con manejo de errores
17/10/2026	La máquina virtual no quería abrir en VirtualBox	Salía como "Inaccesible" por la instantánea que se había trabado	Se quitó el registro de esa instantánea y la VM volvió a abrir con todo mi trabajo	—
17/10/2026	Hice el frontend con HTML y JavaScript usando fetch, y configuré CORS	El navegador no dejaba que la página pidiera datos a la API por estar en otro puerto; además salía "carrito vacío" aunque sí tenía productos	Permití en CORS solo http://localhost:5500 y arreglé el CSS para que el mensaje se oculte	3b4052c feat: agrega frontend HTML y JavaScript con fetch para catalogo y carrito, y configura CORS
17/10/2026	Subí el README y el script de la base de datos	La consigna pide explicar cómo instalar y usar el proyecto	Escribí el README con instalación, configuración, endpoints y limitaciones, y agregué database/cc.sql	cc3e4df docs: agrega README y script SQL de la base de datos
17/10/2026	Agregué la versión PHP original al repositorio	La consigna pide el código de las dos versiones, pero el Config.php tenía la contraseña de MySQL	La copié sin la carpeta vendor y cambié la contraseña por CAMBIAR	279f3e7 docs: agrega version PHP original como referencia, sin vendor ni credenciales
17/10/2026	Hice el diagrama de casos de uso	Ejecuté dos veces el commit y Dia guardó un ajuste del tamaño de los óvalos	Quedaron dos commits; los dejé así para no reescribir el historial	a506579 y 2183564 docs: agrega diagrama de casos de uso
17/10/2026	Hice los diagramas de arquitectura de la versión PHP y la de Spring Boot	Mostrar la diferencia entre las dos versiones	En PHP el servidor arma el HTML; en Spring Boot la API manda JSON y el navegador arma la página	210373e docs: agrega diagramas de arquitectura PHP y Spring Boot por capas

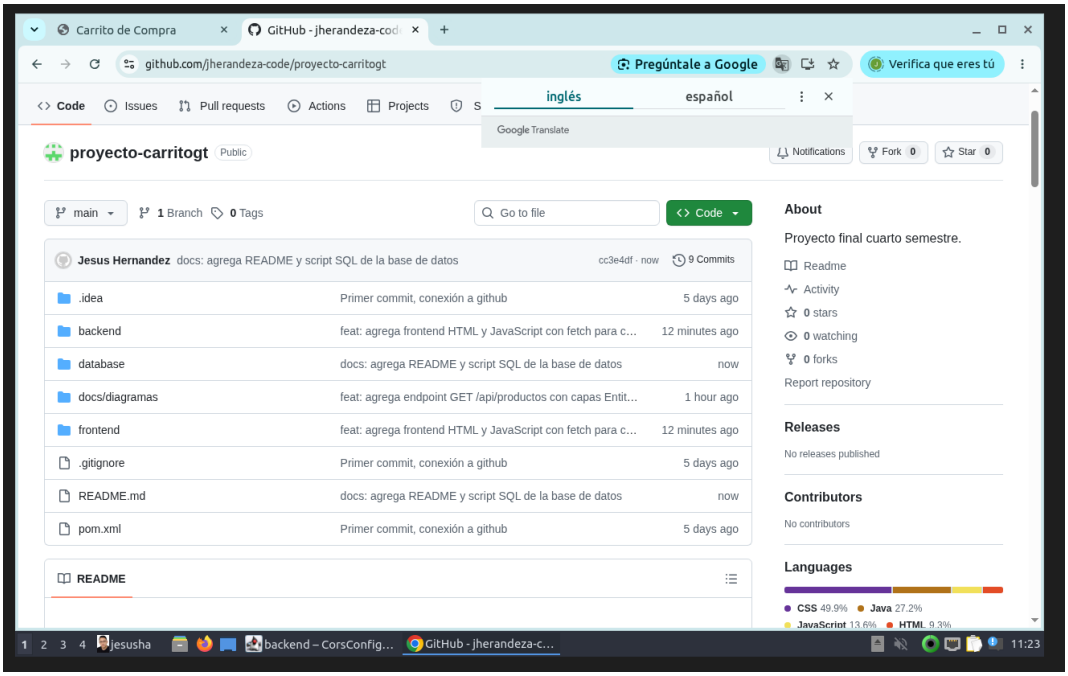


Figura 24. Repositorio en GitHub con las carpetas del proyecto y el historial de commits.